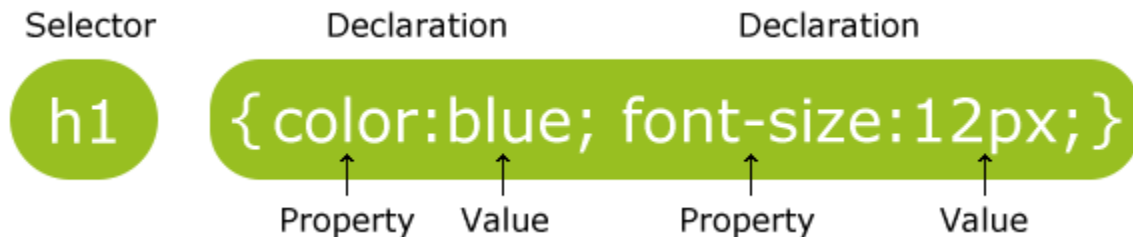


CSS Tutorial

http://www.w3schools.com/css/css_intro.asp

A CSS rule has two main parts: a selector, and one or more declarations:



The selector is normally the HTML element you want to style.

Each declaration consists of a property and a value.

The property is the style attribute you want to change. Each property has a value.

CSS declarations always ends with a semicolon, and declaration groups are surrounded by curly brackets:

```
p {color:red;text-align:center;}
```

To make the CSS more readable, you can put one declaration on each line, like this:

Example

```
p
{
color:red;
text-align:center;
}
```

CSS Comments

Comments are used to explain your code, and may help you when you edit the source code at a later date. Comments are ignored by browsers.

A CSS comment begins with "/*", and ends with "*/", like this:

```
/*This is a comment*/
p
{
text-align:center;
```

```
/*This is another comment*/  
color:black;  
font-family:arial;  
}
```

CSS Selectors

In CSS, selectors are patterns used to select the element(s) you want to style.

Use our [CSS Selector Tester](#) to demonstrate the different selectors.

The "CSS" column indicates in which CSS version the property is defined (CSS1, CSS2, or CSS3).

Selector	Example	Example description	CSS
<u>.class</u>	.intro	Selects all elements with class="intro"	1
<u>#id</u>	#firstname	Selects the element with id="firstname"	1
<u>*</u>	*	Selects all elements	2
<u>element</u>	p	Selects all <p> elements	1
<u>element,element</u>	div, p	Selects all <div> elements and all <p> elements	1
<u>element element</u>	div p	Selects all <p> elements inside <div> elements	1
<u>element>element</u>	div > p	Selects all <p> elements where the parent is a <div> element	2
<u>element+element</u>	div + p	Selects all <p> elements that are placed immediately after <div> elements	2
<u>element1~element2</u>	p ~ ul	Selects every element that are preceded by a <p> element	3
<u>[attribute]</u>	[target]	Selects all elements with a target attribute	2
<u>[attribute=value]</u>	[target=_blank]	Selects all elements with target="_blank"	2
<u>[attribute~=value]</u>	[title~=flower]	Selects all elements with a title attribute containing the word "flower"	2
<u>[attribute =value]</u>	[lang=en]	Selects all elements with a lang attribute value starting with "en"	2

<u>[attribute^=value]</u>	a[href^="https"]	Selects every <a> element whose href attribute value begins with "https"	3
<u>[attribute\$=value]</u>	a[href\$=".pdf"]	Selects every <a> element whose href attribute value ends with ".pdf"	3
<u>[attribute*=value]</u>	a[href*="w3schools"]	Selects every <a> element whose href attribute value contains the substring "w3schools"	3
<u>:active</u>	a:active	Selects the active link	1
<u>::after</u>	p::after	Insert something after the content of each <p> element	2
<u>::before</u>	p::before	Insert something before the content of each <p> element	2
<u>:checked</u>	input:checked	Selects every checked <input> element	3
<u>:disabled</u>	input:disabled	Selects every disabled <input> element	3
<u>:empty</u>	p:empty	Selects every <p> element that has no children (including text nodes)	3
<u>:enabled</u>	input:enabled	Selects every enabled <input> element	3
<u>:first-child</u>	p:first-child	Selects every <p> element that is the first child of its parent	2
<u>::first-letter</u>	p::first-letter	Selects the first letter of every <p> element	1
<u>::first-line</u>	p::first-line	Selects the first line of every <p> element	1
<u>:first-of-type</u>	p:first-of-type	Selects every <p> element that is the first <p> element of its parent	3
<u>:focus</u>	input:focus	Selects the input element which has focus	2
<u>:hover</u>	a:hover	Selects links on mouse over	1
<u>:in-range</u>	input:in-range	Selects input elements with a value within a specified range	3
<u>:invalid</u>	input:invalid	Selects all input elements with an invalid value	3
<u>:lang(<i>language</i>)</u>	p:lang(it)	Selects every <p> element with a lang attribute equal to "it" (Italian)	2
<u>:last-child</u>	p:last-child	Selects every <p> element that is the last child of its parent	3
<u>:last-of-type</u>	p:last-of-type	Selects every <p> element that is the last <p> element of its parent	3
<u>:link</u>	a:link	Selects all unvisited links	1
<u>:not(<i>selector</i>)</u>	:not(p)	Selects every element that is not a <p> element	3
<u>:nth-child(<i>n</i>)</u>	p:nth-child(2)	Selects every <p> element that is the second child of its parent	3
<u>:nth-last-child(<i>n</i>)</u>	p:nth-last-child(2)	Selects every <p> element that is the second child of its parent, counting from the last child	3
<u>:nth-last-of-type(<i>n</i>)</u>	p:nth-last-of-type(2)	Selects every <p> element that is the second <p> element of its parent, counting from the last child	3
<u>:nth-of-type(<i>n</i>)</u>	p:nth-of-type(2)	Selects every <p> element that is the second <p> element of its parent	3

:only-of-type	p:only-of-type	Selects every <p> element that is the only <p> element of its parent	3
:only-child	p:only-child	Selects every <p> element that is the only child of its parent	3
:optional	input:optional	Selects input elements with no "required" attribute	3
:out-of-range	input:out-of-range	Selects input elements with a value outside a specified range	3
:read-only	input:read-only	Selects input elements with the "readonly" attribute specified	3
:read-write	input:read-write	Selects input elements with the "readonly" attribute NOT specified	3
:required	input:required	Selects input elements with the "required" attribute specified	3
:root	:root	Selects the document's root element	3
::selection	::selection	Selects the portion of an element that is selected by a user	
:target	#news:target	Selects the current active #news element (clicked on a URL containing that anchor name)	3
:valid	input:valid	Selects all input elements with a valid value	3
:visited	a:visited	Selects all visited links	1

The id and class Selectors

In addition to setting a style for a HTML element, CSS allows you to specify your own selectors called "id" and "class".

The id Selector

The id selector is used to specify a style for a single, unique element.

The id selector uses the id attribute of the HTML element, and is defined with a "#".

The style rule below will be applied to the element with id="para1":

Example

```
#para1
{
text-align:center;
color:red;
}
```

The class Selector

The class selector is used to specify a style for a group of elements. Unlike the id selector, the class selector is most often used on several elements.

This allows you to set a particular style for any HTML elements with the same class.

The class selector uses the HTML class attribute, and is defined with a ".".

In the example below, all HTML elements with class="center" will be center-aligned:

Example

```
.center {text-align:center;}
```

You can also specify that only specific HTML elements should be affected by a class.

In the example below, all p elements with class="center" will be center-aligned:

Example

```
p.center {text-align:center;}
```

External Style Sheet

An external style sheet is ideal when the style is applied to many pages. With an external style sheet, you can change the look of an entire Web site by changing one file. Each page must link to the style sheet using the <link> tag. The <link> tag goes inside the head section:

```
<head>
<link rel="stylesheet" type="text/css" href="mystyle.css" />
</head>
```

An external style sheet can be written in any text editor. The file should not contain any html tags. Your style sheet should be saved with a .css extension. An example of a style sheet file is shown below:

```
hr {color:sienna;}
p {margin-left:20px;}
body {background-image:url("images/back40.gif");}
```

Internal Style Sheet

An internal style sheet should be used when a single document has a unique style. You define internal styles in the head section of an HTML page, by using the <style> tag, like this:

```
<head>
<style type="text/css">
hr {color:sienna;}
p {margin-left:20px;}
body {background-image:url("images/back40.gif");}
</style>
</head>
```

Inline Styles

An inline style loses many of the advantages of style sheets by mixing content with presentation. Use this method sparingly!

To use inline styles you use the style attribute in the relevant tag. The style attribute can contain any CSS property. The example shows how to change the color and the left margin of a paragraph:

```
<p style="color:sienna;margin-left:20px">This is a paragraph.</p>
```

Multiple Style Sheets

If some properties have been set for the same selector in different style sheets, the values will be inherited from the more specific style sheet.

For example, an external style sheet has these properties for the h3 selector:

```
h3
{
color:red;
text-align:left;
font-size:8pt;
}
```

And an internal style sheet has these properties for the h3 selector:

```
h3
{
text-align:right;
font-size:20pt;
}
```

If the page with the internal style sheet also links to the external style sheet the properties for h3 will be:

```
color:red;
text-align:right;
font-size:20pt;
```

The color is inherited from the external style sheet and the text-alignment and the font-size is replaced by the internal style sheet.

Multiple Styles Will Cascade into One

Styles can be specified:

- inside an HTML element
- inside the head section of an HTML page
- in an external CSS file

Tip: Even multiple external style sheets can be referenced inside a single HTML document.

Cascading order

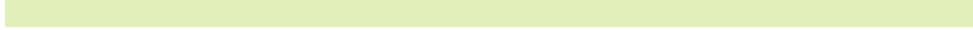
What style will be used when there is more than one style specified for an HTML element?

Generally speaking we can say that all the styles will "cascade" into a new "virtual" style sheet by the following rules, where number four has the highest priority:

1. Browser default
2. External style sheet
3. Internal style sheet (in the head section)
4. Inline style (inside an HTML element)

So, an inline style (inside an HTML element) has the highest priority, which means that it will override a style defined inside the <head> tag, or in an external style sheet, or in a browser (a default value).

Note: If the link to the external style sheet is placed after the internal style sheet in HTML <head>, the external style sheet will override the internal style sheet!



CSS background properties are used to define the background effects of an element.

CSS properties used for background effects:

- background-color
- background-image
- background-repeat
- background-attachment
- background-position

Background Color

The background-color property specifies the background color of an element.

The background color of a page is defined in the body selector:

Example

```
body {background-color:#b0c4de;}
```

The background color can be specified by:

- name - a color name, like "red"
- RGB - an RGB value, like "rgb(255,0,0)"
- Hex - a hex value, like "#ff0000"

In the example below, the h1, p, and div elements have different background colors:

Example

```
h1 {background-color:#6495ed;}  
p {background-color:#e0ffff;}  
div {background-color:#b0c4de;}
```

Background Image

The background-image property specifies an image to use as the background of an element.

By default, the image is repeated so it covers the entire element.

The background image for a page can be set like this:

Example

```
body {background-image:url('paper.gif');}
```

Background Image - Repeat Horizontally or Vertically

By default, the background-image property repeats an image both horizontally and vertically.

Some images should be repeated only horizontally or vertically, or they will look strange, like this:

If the image is repeated only horizontally (repeat-x), the background will look better:

Example

```
body
{
background-image:url('gradient2.png');
background-repeat:repeat-x;
}
```

Background Image - Set position and no-repeat

💡 When using a background image, use an image that does not disturb the text.

Showing the image only once is specified by the background-repeat property:

Example

```
body
{
background-image:url('img_tree.png');
background-repeat:no-repeat;
}
```

In the example above, the background image is shown in the same place as the text. We want to change the position of the image, so that it does not disturb the text too much.

The position of the image is specified by the background-position property:

Example

```
body
{
background-image:url('img_tree.png');
background-repeat:no-repeat;
background-position:right top;
margin-right:200px;
}
```

Background - Shorthand property

As you can see from the examples above, there are many properties to consider when dealing with backgrounds.

To shorten the code, it is also possible to specify all the properties in one single property. This is called a shorthand property.

The shorthand property for background is simply "background":

Example

```
body {background:#ffffff url('img_tree.png') no-repeat right top;}
```

When using the shorthand property the order of the property values are:

- background-color
- background-image
- background-repeat
- background-attachment
- background-position

It does not matter if one of the property values are missing, as long as the ones that are present are in this order.

All CSS Background Properties

The number in the "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Property	Description	Values	CSS
background	Sets all the background properties in one declaration	<i>background-color</i> <i>background-image</i> <i>background-repeat</i> <i>background-attachment</i> <i>background-position</i> inherit	1
background-attachment	Sets whether a background image is fixed or scrolls with the rest of the page	scroll fixed inherit	1
background-color	Sets the background color of an element	<i>color-rgb</i> <i>color-hex</i> <i>color-name</i> transparent inherit	1
background-image	Sets the background image for an element	url(<i>URL</i>) none inherit	1
background-position	Sets the starting position of a background image	left top left center left bottom right top right center right bottom center top center center center bottom <i>x% y%</i> <i>xpos ypos</i> inherit	1
background-repeat	Sets if/how a background image will be repeated	repeat repeat-x	1

		repeat-y no-repeat inherit	
--	--	----------------------------------	--

Text Alignment

The text-align property is used to set the horizontal alignment of a text.

Text can be centered, or aligned to the left or right, or justified.

When text-align is set to "justify", each line is stretched so that every line has equal width, and the left and right margins are straight (like in magazines and newspapers).

Example

```
h1 {text-align:center;}
p.date {text-align:right;}
p.main {text-align:justify;}
```

Text Decoration

The text-decoration property is used to set or remove decorations from text.

The text-decoration property is mostly used to remove underlines from links for design purposes:

Example

```
a {text-decoration:none;}
```

It can also be used to decorate text:

Example

```
h1 {text-decoration:overline;}
h2 {text-decoration:line-through;}
h3 {text-decoration:underline;}
h4 {text-decoration:blink;}
```

Text Transformation

The text-transform property is used to specify uppercase and lowercase letters in a text.

It can be used to turn everything into uppercase or lowercase letters, or capitalize the first letter of each word.

Example

```
p.uppercase {text-transform:uppercase;}
p.lowercase {text-transform:lowercase;}
```

```
p.capitalize {text-transform:capitalize;}
```

```
<html>
<head>
<style type="text/css">
p.uppercase {text-transform:uppercase;}
p.lowercase {text-transform:lowercase;}
p.capitalize {text-transform:capitalize;}
</style>
</head>

<body>
<p class="uppercase">This is some text.</p>
<p class="lowercase">This is some text.</p>
<p class="capitalize">This is some text.</p>
</body>
</html>
```

Text Indentation

The text-indentation property is used to specify the indentation of the first line of a text.

Example

```
p {text-indent:50px;}
```

All CSS Text Properties

The number in the "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Property	Description	Values	CSS
color	Sets the color of a text	<i>color</i>	1
direction	Sets the text direction	ltr rtl	2
line-height	Sets the distance between lines	normal <i>number</i> <i>length</i> %	1
letter-spacing	Increase or decrease the space between characters	normal <i>length</i>	1
text-align	Aligns the text in an element	left right center justify	1
text-decoration	Adds decoration to text	none underline overline line-through blink	1
text-indent	Indents the first line of text in an element	<i>length</i> %	1

text-shadow		none color length	
text-transform	Controls the letters in an element	none capitalize uppercase lowercase	1
unicode-bidi		normal embed bidi-override	2
vertical-align	Sets the vertical alignment of an element	baseline sub super top text-top middle bottom text-bottom length %	1
white-space	Sets how white space inside an element is handled	normal pre nowrap	1
word-spacing	Increase or decrease the space between words	normal length	1

CSS Font Families

In CSS, there are two types of font family names:

- **generic family** - a group of font families with a similar look (like "Serif" or "Monospace")
- **font family** - a specific font family (like "Times New Roman" or "Arial")

Generic family	Font family	Description
Serif	Times New Roman Georgia	Serif fonts have small lines at the ends on some characters
Sans-serif	Arial Verdana	"Sans" means without - these fonts do not have the lines at the ends of characters
Monospace	Courier New Lucida Console	All monospace characters has the same width

Font Family

The font family of a text is set with the font-family property.

The font-family property should hold several font names as a "fallback" system. If the browser does not support the first font, it tries the next font.

Start with the font you want, and end with a generic family, to let the browser pick a similar font in the generic family, if no other fonts are available.

Note: If the name of a font family is more than one word, it must be in quotation marks, like font-family: "Times New Roman".

More than one font family is specified in a comma-separated list:

Example

```
p{font-family:"Times New Roman", Times, serif;}
```

Font Style

The font-style property is mostly used to specify italic text.

This property has three values:

- normal - The text is shown normally
- italic - The text is shown in italics
- oblique - The text is "leaning" (oblique is very similar to italic, but less supported)

Example

```
p.normal {font-style:normal;}  
p.italic {font-style:italic;}  
p.oblique {font-style:oblique;}
```

Font Size

The font-size property sets the size of the text.

Being able to manage the text size is important in web design. However, you should not use font size adjustments to make paragraphs look like headings, or headings look like paragraphs.

Always use the proper HTML tags, like <h1> - <h6> for headings and <p> for paragraphs.

The font-size value can be an absolute, or relative size.

Absolute size:

- Sets the text to a specified size
- Does not allow a user to change the text size in all browsers (bad for accessibility reasons)
- Absolute size is useful when the physical size of the output is known

Relative size:

- Sets the size relative to surrounding elements
- Allows a user to change the text size in browsers

 If you do not specify a font size, the default size for normal text, like paragraphs, is 16px (16px=1em).

Set Font Size With Pixels

Setting the text size with pixels, gives you full control over the text size:

Example

```
h1 {font-size:40px;}
h2 {font-size:30px;}
p {font-size:14px;}
```

Set Font Size With Em

To avoid the resizing problem with Internet Explorer, many developers use em instead of pixels.

The em size unit is recommended by the W3C.

1em is equal to the current font size. The default text size in browsers is 16px. So, the default size of 1em is 16px.

The size can be calculated from pixels to em using this formula: $pixels/16=em$

Example

```
h1 {font-size:2.5em;} /* 40px/16=2.5em */
h2 {font-size:1.875em;} /* 30px/16=1.875em */
p {font-size:0.875em;} /* 14px/16=0.875em */
```

[Try it yourself >>](#)

In the example above, the text size in em is the same as the previous example in pixels. However, with the em size, it is possible to adjust the text size in all browsers.

Unfortunately, there is still a problem with IE. When resizing the text, it becomes larger than it should when made larger, and smaller than it should when made smaller.

Use a Combination of Percent and Em

The solution that works in all browsers, is to set a default font-size in percent for the body element:

Example

```
body {font-size:100%;}
h1 {font-size:2.5em;}
h2 {font-size:1.875em;}
p {font-size:0.875em;}
```

[Try it yourself >>](#)

Our code now works great! It shows the same text size in all browsers, and allows all browsers to zoom or resize the text!

All CSS Font Properties

The number in the "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Property	Description	Values	CSS
font	Sets all the font properties in one declaration	<i>font-style</i> <i>font-variant</i> <i>font-weight</i> <i>font-size/line-height</i> <i>font-family</i> caption icon menu message-box small-caption status-bar inherit	1
font-family	Specifies the font family for text	<i>family-name</i> <i>generic-family</i> inherit	1
font-size	Specifies the font size of text	xx-small x-small small medium large x-large xx-large smaller larger <i>length</i> % inherit	1
font-style	Specifies the font style for text	normal italic oblique inherit	1
font-variant	Specifies whether or not a text should be displayed in a small-caps font	normal small-caps inherit	1
font-weight	Specifies the weight of a font	normal bold bolder lighter 100 200 300 400 500 600 700 800 900 inherit	1

Styling Links

Links can be style with any CSS property (e.g. color, font-family, background-color).

Special for links are that they can be styled differently depending on what state they are in.

The four links states are:

- a:link - a normal, unvisited link
- a:visited - a link the user has visited
- a:hover - a link when the user mouses over it
- a:active - a link the moment it is clicked

Example

```
a:link {color:#FF0000;}      /* unvisited link */
a:visited {color:#00FF00;}  /* visited link */
a:hover {color:#FF00FF;}    /* mouse over link */
a:active {color:#0000FF;}    /* selected link */
```

[Try it yourself »](#)

When setting the style for several link states, there are some order rules:

- a:hover MUST come after a:link and a:visited
- a:active MUST come after a:hover

Common Link Styles

In the example above the link changes color depending on what state it is in.

Lets go through some of the other common ways to style links:

Text Decoration

The text-decoration property is mostly used to remove underlines from links:

Example

```
a:link {text-decoration:none;}
a:visited {text-decoration:none;}
a:hover {text-decoration:underline;}
a:active {text-decoration:underline;}
```

[Try it yourself »](#)

Background Color

The background-color property specifies the background color for links:

Example

```
a:link {background-color:#B2FF99;}
a:visited {background-color:#FFFF85;}
```

```
a:hover {background-color:#FF704D;}
a:active {background-color:#FF704D;}
```

The CSS list properties allow you to:

- Set different list item markers for ordered lists
- Set different list item markers for unordered lists
- Set an image as the list item marker

List

In HTML, there are two types of lists:

- unordered lists - the list items are marked with bullets
- ordered lists - the list items are marked with numbers or letters

With CSS, lists can be styled further, and images can be used as the list item marker.

Different List Item Markers

The type of list item marker is specified with the `list-style-type` property:

Example

```
ul.a {list-style-type: circle;}
ul.b {list-style-type: square;}

ol.c {list-style-type: upper-roman;}
ol.d {list-style-type: lower-alpha;}
```

[Try it yourself »](#)

Some of the property values are for unordered lists, and some for ordered lists.

Values for Unordered Lists

Value	Description
none	No marker
disc	Default. The marker is a filled circle
circle	The marker is a circle
square	The marker is a square

Values for Ordered Lists

Value	Description
armenian	The marker is traditional Armenian numbering
decimal	The marker is a number
decimal-leading-zero	The marker is a number padded by initial zeros (01, 02, 03, etc.)

georgian	The marker is traditional Georgian numbering (an, ban, gan, etc.)
lower-alpha	The marker is lower-alpha (a, b, c, d, e, etc.)
lower-greek	The marker is lower-greek (alpha, beta, gamma, etc.)
lower-latin	The marker is lower-latin (a, b, c, d, e, etc.)
lower-roman	The marker is lower-roman (i, ii, iii, iv, v, etc.)
upper-alpha	The marker is upper-alpha (A, B, C, D, E, etc.)
upper-latin	The marker is upper-latin (A, B, C, D, E, etc.)
upper-roman	The marker is upper-roman (I, II, III, IV, V, etc.)

 **Note:** No versions of Internet Explorer (including IE8) support the property values "decimal-leading-zero", "lower-greek", "lower-latin", "upper-latin", "armenian", or "georgian".

An Image as The List Item Marker

To specify an image as the list item marker, use the `list-style-image` property:

Example

```
ul
{
list-style-image: url('sqpurple.gif');
}
```

[Try it yourself »](#)

The example above does not display equally in all browsers. IE and Opera will display the image-marker a little bit higher than Firefox, Chrome, and Safari.

If you want the image-marker to be placed equally in all browsers, a crossbrowser solution is explained below.

Crossbrowser Solution

The following example displays the image-marker equally in all browsers:

Example

```
ul
{
list-style-type: none;
padding: 0px;
margin: 0px;
}
li
{
background-image: url(sqpurple.gif);
background-repeat: no-repeat;
background-position: 0px 5px;
padding-left: 14px;
}
```

[Try it yourself »](#)

Example explained:

- For ul:
 - Set the list-style-type to none to remove the list item marker
 - Set both padding and margin to 0px (for cross-browser compatibility)
- For li:
 - Set the URL of the image, and show it only once (no-repeat)
 - Position the image where you want it (left 0px and down 5px)
 - Position the text in the list with padding-left

List - Shorthand property

It is also possible to specify all the list properties in one, single property. This is called a shorthand property.

The shorthand property used for lists, is the list-style property:

Example

```
ul
{
list-style: square url("sqpurple.gif");
}
```

Try it yourself »

When using the shorthand property, the order of the values are:

- list-style-type
- list-style-position (for a description, see the CSS properties table below)
- list-style-image

It does not matter if one of the values above are missing, as long as the rest are in the specified order.



More Examples

[All the different list-item markers for lists](#)

This example demonstrates all the different list-item markers in CSS.

All CSS List Properties

The number in the "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Property	Description	Values	CSS
list-style	Sets all the properties for a list in one declaration	<i>list-style-type</i> <i>list-style-position</i>	1

		<i>list-style-image</i> inherit	
<u>list-style-image</u>	Specifies an image as the list-item marker	URL none inherit	1
<u>list-style-position</u>	Specifies if the list-item markers should appear inside or outside the content flow	inside outside inherit	1
<u>list-style-type</u>	Specifies the type of list-item marker	none disc circle square decimal decimal-leading-zero armenian georgian lower-alpha upper-alpha lower-greek lower-latin upper-latin lower-roman upper-roman inherit	1

The look of an HTML table can be greatly improved with CSS:

Company	Contact	Country
Alfreds Futterkiste	Maria Anders	Germany
Berglunds snabbköp	Christina Berglund	Sweden
Centro comercial Moctezuma	Francisco Chang	Mexico
Ernst Handel	Roland Mendel	Austria
Island Trading	Helen Bennett	UK
Königlich Essen	Philip Cramer	Germany
Laughing Bacchus Winecellars	Yoshi Tannamuri	Canada
Magazzini Alimentari Riuniti	Giovanni Rovelli	Italy
North/South	Simon Crowther	UK
Paris spécialités	Marie Bertrand	France
The Big Cheese	Liz Nixon	USA
Vaffeljernet	Palle Ibsen	Denmark

Table Borders

To specify table borders in CSS, use the border property.

The example below specifies a black border for table, th, and td elements:

Example

```
table, th, td
```

```
{  
border: 1px solid black;  
}
```

[Try it yourself »](#)

Notice that the table in the example above has double borders. This is because both the table, th, and td elements have separate borders.

To display a single border for the table, use the border-collapse property.

Collapse Borders

The border-collapse property sets whether the table borders are collapsed into a single border or separated:

Example

```
table  
{  
border-collapse: collapse;  
}  
table, th, td  
{  
border: 1px solid black;  
}
```

[Try it yourself »](#)

Table Width and Height

Width and height of a table is defined by the width and height properties.

The example below sets the width of the table to 100%, and the height of the th elements to 50px:

Example

```
table  
{  
width: 100%;  
}  
th  
{  
height: 50px;  
}
```

[Try it yourself »](#)

Table Text Alignment

The text in a table is aligned with the text-align and vertical-align properties.

The text-align property sets the horizontal alignment, like left, right, or center:

Example

```
td
{
text-align:right;
}
```

[Try it yourself »](#)

The vertical-align property sets the vertical alignment, like top, bottom, or middle:

Example

```
td
{
height:50px;
vertical-align:bottom;
}
```

[Try it yourself »](#)

Table Padding

To control the space between the border and content in a table, use the padding property on td and th elements:

Example

```
td
{
padding:15px;
}
```

[Try it yourself »](#)

Table Color

The example below specifies the color of the borders, and the text and background color of th elements:

Example

```
table, td, th
{
border:1px solid green;
}
th
{
background-color:green;
color:white;
}
```

The CSS Box Model

All HTML elements can be considered as boxes. In CSS, the term "box model" is used when talking about design and layout.

The CSS box model is essentially a box that wraps around HTML elements, and it consists of: margins, borders, padding, and the actual content.

The box model allows us to place a border around elements and space elements in relation to other elements.

The image below illustrates the box model:



Explanation of the different parts:

- **Margin** - Clears an area around the border. The margin does not have a background color, and it is completely transparent
- **Border** - A border that lies around the padding and content. The border is affected by the background color of the box
- **Padding** - Clears an area around the content. The padding is affected by the background color of the box
- **Content** - The content of the box, where text and images appear

In order to set the width and height of an element correctly in all browsers, you need to know how the box model works.

Width and Height of an Element

Important: When you specify the width and height properties of an element with CSS, you are just setting the width and height of the content area. To know the full size of the element, you must also add the padding, border and margin.

The total width of the element in the example below is 300px:

```
width:250px;
padding:10px;
border:5px solid gray;
margin:10px;
```

Let's do the math:

250px (width)
+ 20px (left and right padding)
+ 10px (left and right border)
+ 20px (left and right margin)
= 300px

Imagine that you only had 250px of space. Let's make an element with a total width of 250px:

Example

```
width:220px;
padding:10px;
border:5px solid gray;
margin:0px;
```

[Try it yourself >](#)

The total width of an element should always be calculated like this:

Total element width = width + left padding + right padding + left border + right border + left margin + right margin

The total height of an element should always be calculated like this:

Total element height = height + top padding + bottom padding + top border + bottom border + top margin + bottom margin

Browsers Compatibility Issue

If you tested the previous example in Internet Explorer, you saw that the total width was not exactly 250px.

IE includes padding and border in the width, when the width property is set, **unless a DOCTYPE is declared**.

To fix this problem, just add a DOCTYPE to the code:

Example

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html>
<head>
<style type="text/css">
div.ex
{
```

```
width:220px;
padding:10px;
border:5px solid gray;
margin:0px;
}
</style>
</head>
```

CSS Border

[« Previous](#)

[Next Chapter »](#)

CSS Border Properties

The CSS border properties allow you to specify the style and color of an element's border.

Border Style

The border-style property specifies what kind of border to display.



None of the border properties will have ANY effect unless the **border-style** property is set!

border-style values:

none: Defines no border

dotted: Defines a dotted border

dashed: Defines a dashed border

solid: Defines a solid border

double: Defines two borders. The width of the two borders are the same as the border-width value

groove: Defines a 3D grooved border. The effect depends on the border-color value

ridge: Defines a 3D ridged border. The effect depends on the border-color value

inset: Defines a 3D inset border. The effect depends on the border-color value

outset: Defines a 3D outset border. The effect depends on the border-color value

Try it yourself: [Set the style of the border](#)

Border Width

The border-width property is used to set the width of the border.

The width is set in pixels, or by using one of the three pre-defined values: thin, medium, or thick.

Note: The "border-width" property does not work if it is used alone. Use the "border-style" property to set the borders first.

Example

```
p.one
{
border-style:solid;
border-width:5px;
}
p.two
{
border-style:solid;
border-width:medium;
}
```

[Try it yourself »](#)

Border Color

The border-color property is used to set the color of the border. The color can be set by:

- name - specify a color name, like "red"
- RGB - specify a RGB value, like "rgb(255,0,0)"
- Hex - specify a hex value, like "#ff0000"

You can also set the border color to "transparent".

Note: The "border-color" property does not work if it is used alone. Use the "border-style" property to set the borders first.

Example

```
p.one
{
border-style:solid;
border-color:red;
}
p.two
{
border-style:solid;
border-color:#98bf21;
}
```

[Try it yourself »](#)

Border - Individual sides

In CSS it is possible to specify different borders for different sides:

Example

```
p
{
border-top-style:dotted;
border-right-style:solid;
border-bottom-style:dotted;
border-left-style:solid;
}
```

[Try it yourself »](#)

The example above can also be set with a single property:

Example

```
border-style:dotted solid;
```

[Try it yourself »](#)

The border-style property can have from one to four values.

- **border-style:dotted solid double dashed;**
 - top border is dotted
 - right border is solid
 - bottom border is double
 - left border is dashed
- **border-style:dotted solid double;**
 - top border is dotted
 - right and left borders are solid
 - bottom border is double
- **border-style:dotted solid;**
 - top and bottom borders are dotted
 - right and left borders are solid
- **border-style:dotted;**
 - all four borders are dotted

The border-style property is used in the example above. However, it also works with border-width and border-color.

Border - Shorthand property

As you can see from the examples above, there are many properties to consider when dealing with borders.

To shorten the code, it is also possible to specify all the border properties in one property. This is called a shorthand property.

The shorthand property for the border properties is "border":

Example

```
border:5px solid red;
```

[Try it yourself »](#)

When using the border property, the order of the values are:

- border-width
- border-style
- border-color

It does not matter if one of the values above are missing (although, border-style is required), as long as the rest are in the specified order.



More Examples

[All the top border properties in one declaration](#)

This example demonstrates a shorthand property for setting all of the properties for the top border in one declaration.

[Set the style of the bottom border](#)

This example demonstrates how to set the style of the bottom border.

[Set the width of the left border](#)

This example demonstrates how to set the width of the left border.

[Set the color of the four borders](#)

This example demonstrates how to set the color of the four borders. It can have from one to four colors.

[Set the color of the right border](#)

This example demonstrates how to set the color of the right border.

All CSS Border Properties

The number in the "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Property	Description	Values	CSS
border	Sets all the border properties in one declaration	<i>border-width</i> <i>border-style</i> <i>border-color</i>	1
border-bottom	Sets all the bottom border properties in one declaration	<i>border-bottom-width</i> <i>border-bottom-style</i> <i>border-bottom-color</i>	1
border-bottom-color	Sets the color of the bottom border	<i>border-color</i>	2
border-bottom-style	Sets the style of the bottom border	<i>border-style</i>	2
border-bottom-width	Sets the width of the bottom border	<i>border-width</i>	1
border-color	Sets the color of the four borders	<i>color_name</i> <i>hex_number</i>	1

		<i>rgb_number</i> <i>transparent</i> <i>inherit</i>	
<u>border-left</u>	Sets all the left border properties in one declaration	<i>border-left-width</i> <i>border-left-style</i> <i>border-left-color</i>	1
<u>border-left-color</u>	Sets the color of the left border	<i>border-color</i>	2
<u>border-left-style</u>	Sets the style of the left border	<i>border-style</i>	2
<u>border-left-width</u>	Sets the width of the left border	<i>border-width</i>	1
<u>border-right</u>	Sets all the right border properties in one declaration	<i>border-right-width</i> <i>border-right-style</i> <i>border-right-color</i>	1
<u>border-right-color</u>	Sets the color of the right border	<i>border-color</i>	2
<u>border-right-style</u>	Sets the style of the right border	<i>border-style</i>	2
<u>border-right-width</u>	Sets the width of the right border	<i>border-width</i>	1
<u>border-style</u>	Sets the style of the four borders	<i>none</i> <i>hidden</i> <i>dotted</i> <i>dashed</i> <i>solid</i> <i>double</i> <i>groove</i> <i>ridge</i> <i>inset</i> <i>outset</i> <i>inherit</i>	1
<u>border-top</u>	Sets all the top border properties in one declaration	<i>border-top-width</i> <i>border-top-style</i> <i>border-top-color</i>	1
<u>border-top-color</u>	Sets the color of the top border	<i>border-color</i>	2
<u>border-top-style</u>	Sets the style of the top border	<i>border-style</i>	2
<u>border-top-width</u>	Sets the width of the top border	<i>border-width</i>	1
<u>border-width</u>	Sets the width of the four borders	<i>thin</i> <i>medium</i> <i>thick</i> <i>length</i> <i>inherit</i>	1

CSS Outlines

[« Previous](#)

[Next Chapter »](#)

An outline is a line that is drawn around elements, outside the border edge, to make the element "stand out".

The outline properties specifies the style, color, and width of an outline.

Examples

[Draw a line around an element \(outline\)](#)

This example demonstrates how to draw a line around an element, outside the border edge.

[Set the style of an outline](#)

This example demonstrates how to set the style of an outline.

[Set the color of an outline](#)

This example demonstrates how to set the color of an outline.

[Set the width of an outline](#)

This example demonstrates how to set the width of an outline.

All CSS Outline Properties

The number in the "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Property	Description	Values	CSS
outline	Sets all the outline properties in one declaration	<i>outline-color</i> <i>outline-style</i> <i>outline-width</i> inherit	2
outline-color	Sets the color of an outline	<i>color_name</i> <i>hex_number</i> <i>rgb_number</i> invert inherit	2
outline-style	Sets the style of an outline	none dotted dashed solid double groove ridge inset outset inherit	2
outline-width	Sets the width of an outline	thin medium thick <i>length</i> inherit	2

CSS Margin

[« Previous](#)

[Next Chapter »](#)

The CSS margin properties define the space around elements.

Margin

The margin clears an area around an element (outside the border). The margin does not have a background color, and is completely transparent.

The top, right, bottom, and left margin can be changed independently using separate properties. A shorthand margin property can also be used, to change all margins at once.

Possible Values

Value	Description
auto	The browser sets the margin. The result of this is dependant of the browser
<i>length</i>	Defines a fixed margin (in pixels, pt, em, etc.)
%	Defines a margin in % of the containing element



It is possible to use negative values, to overlap content.

Margin - Individual sides

In CSS, it is possible to specify different margins for different sides:

Example

```
margin-top:100px;  
margin-bottom:100px;  
margin-right:50px;  
margin-left:50px;
```

[Try it yourself »](#)

Margin - Shorthand property

To shorten the code, it is possible to specify all the margin properties in one property. This is called a shorthand property.

The shorthand property for all the margin properties is "margin":

Example

```
margin:100px 50px;
```

[Try it yourself »](#)

The margin property can have from one to four values.

- **margin:25px 50px 75px 100px;**
 - top margin is 25px
 - right margin is 50px
 - bottom margin is 75px
 - left margin is 100px
- **margin:25px 50px 75px;**
 - top margin is 25px
 - right and left margins are 50px
 - bottom margin is 75px

- **margin:25px 50px;**
 - top and bottom margins are 25px
 - right and left margins are 50px
- **margin:25px;**
 - all four margins are 25px



More Examples

[Set the top margin of a text using a cm value](#)

This example demonstrates how to set the top margin of a text using a cm value.

[Set the bottom margin of a text using a percent value](#)

This example demonstrates how to set the bottom margin of a text using a percent value.

All CSS Margin Properties

The number in the "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Property	Description	Values	CSS
margin	A shorthand property for setting the margin properties in one declaration	<i>margin-top</i> <i>margin-right</i> <i>margin-bottom</i> <i>margin-left</i>	1
margin-bottom	Sets the bottom margin of an element	auto <i>length</i> %	1
margin-left	Sets the left margin of an element	auto <i>length</i> %	1
margin-right	Sets the right margin of an element	auto <i>length</i> %	1
margin-top	Sets the top margin of an element	auto <i>length</i> %	1

CSS Padding

[« Previous](#)

[Next Chapter »](#)

The CSS padding properties define the space between the element border and the element content.

Padding

The padding clears an area around the content (inside the border) of an element. The padding is affected by the background color of the element.

The top, right, bottom, and left padding can be changed independently using separate properties. A shorthand padding property can also be used, to change all paddings at once.

Possible Values

Value	Description
<i>length</i>	Defines a fixed padding (in pixels, pt, em, etc.)
%	Defines a padding in % of the containing element

Padding - Individual sides

In CSS, it is possible to specify different padding for different sides:

Example

```
padding-top:25px;  
padding-bottom:25px;  
padding-right:50px;  
padding-left:50px;
```

[Try it yourself »](#)

Padding - Shorthand property

To shorten the code, it is possible to specify all the padding properties in one property. This is called a shorthand property.

The shorthand property for all the padding properties is "padding":

Example

```
padding:25px 50px;
```

[Try it yourself »](#)

The padding property can have from one to four values.

- **padding:25px 50px 75px 100px;**
 - top padding is 25px
 - right padding is 50px
 - bottom padding is 75px
 - left padding is 100px
- **padding:25px 50px 75px;**
 - top padding is 25px
 - right and left paddings are 50px
 - bottom padding is 75px

- **padding:25px 50px;**
 - top and bottom paddings are 25px
 - right and left paddings are 50px
- **padding:25px;**
 - all four paddings are 25px



More Examples

All the padding properties in one declaration

This example demonstrates a shorthand property for setting all of the padding properties in one declaration, can have from one to four values.

Set the left padding

This example demonstrates how to set the left padding of a p element.

Set the right padding

This example demonstrates how to set the right padding of a p element.

Set the top padding

This example demonstrates how to set the top padding of a p element.

Set the bottom padding

This example demonstrates how to set the bottom padding of a p element.

All CSS Padding Properties

The number in the "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Property	Description	Values	CSS
<u>padding</u>	A shorthand property for setting all the padding properties in one declaration	<i>padding-top</i> <i>padding-right</i> <i>padding-bottom</i> <i>padding-left</i>	1
<u>padding-bottom</u>	Sets the bottom padding of an element	<i>length</i> <i>%</i>	1
<u>padding-left</u>	Sets the left padding of an element	<i>length</i> <i>%</i>	1
<u>padding-right</u>	Sets the right padding of an element	<i>length</i> <i>%</i>	1
<u>padding-top</u>	Sets the top padding of an element	<i>length</i> <i>%</i>	1

CSS Grouping and Nesting Selectors

[<< Previous](#)

[Next Chapter >>](#)

Grouping Selectors

In style sheets there are often elements with the same style.

```
h1
{
color:green;
}
h2
{
color:green;
}
p
{
color:green;
}
```

To minimize the code, you can group selectors.

Separate each selector with a comma.

In the example below we have grouped the selectors from the code above:

Example

```
h1,h2,p
{
color:green;
}
```

[Try it yourself >>](#)

Nesting Selectors

It is possible to apply a style for a selector within a selector.

In the example below, one style is specified for all p elements, and a separate style is specified for p elements nested within the "marked" class:

Example

```
p
{
color:blue;
text-align:center;
}
.marked
{
background-color:blue;
```

```
}  
.marked p  
{  
color:white;  
}
```

CSS Dimension

[« Previous](#)

[Next Chapter »](#)

The CSS dimension properties allow you to control the height and width of an element.



Try it Yourself - Examples

[Set the height of elements](#)

This example demonstrates how to set the height of different elements.

[Set the height of an image using percent](#)

This example demonstrates how to set the height of an element using a percent value.

[Set the width of an element using a pixel value](#)

This example demonstrates how to set the width of an element using a pixel value.

[Set the maximum height of an element](#)

This example demonstrates how to set the maximum height of an element.

[Set the maximum width of an element using percent](#)

This example demonstrates how to set the maximum width of an element using a percent value.

[Set the minimum height of an element](#)

This example demonstrates how to set the minimum height of an element.

[Set the minimum width of an element using a pixel value](#)

This example demonstrates how to set the minimum width of an element using a pixel value.

All CSS Dimension Properties

The number in the "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Property	Description	Values	CSS
height	Sets the height of an element	auto <i>length</i> % inherit	1
max-height	Sets the maximum height of an element	none <i>length</i> % inherit	2

<u>max-width</u>	Sets the maximum width of an element	none <i>length</i> % inherit	2
<u>min-height</u>	Sets the minimum height of an element	<i>length</i> % inherit	2
<u>min-width</u>	Sets the minimum width of an element	<i>length</i> % inherit	2
<u>width</u>	Sets the width of an element	auto <i>length</i> % inherit	1

CSS Display and Visibility

[« Previous](#)

[Next Chapter »](#)

The display property specifies if/how an element is displayed, and the visibility property specifies if an element should be visible or hidden.

Box 1



Box 2



Box 3



Hiding an Element - display:none or visibility:hidden

Hiding an element can be done by setting the display property to "none" or the visibility property to "hidden". However, notice that these two methods produce different results:

visibility:hidden hides an element, but it will still take up the same space as before. The element will be hidden, but still affect the layout.

Example

```
h1.hidden {visibility:hidden;}
```

[Try it yourself »](#)

`display:none` hides an element, and it will not take up any space. The element will be hidden, and the page will be displayed as the element is not there:

Example

```
h1.hidden {display:none;}
```

[Try it yourself »](#)

CSS Display - Block and Inline Elements

A block element is an element that takes up the full width available, and has a line break before and after it.

Examples of block elements:

- `<h1>`
- `<p>`
- `<div>`

An inline element only takes up as much width as necessary, and does not force line breaks.

Examples of inline elements:

- `<span>`
- `<a>`

Changing How an Element is Displayed

Changing an inline element to a block element, or vice versa, can be useful for making the page look a specific way, and still follow web standards.

The following example displays list items as inline elements:

Example

```
li {display:inline;}
```

[Try it yourself »](#)

The following example displays span elements as block elements:

Example

```
span {display:block;}
```

[Try it yourself »](#)

Note: Changing the display type of an element changes only how the element is displayed, NOT what kind of element it is. For example: An inline element set to display:block is not allowed to have a block element nested inside of it.

CSS Positioning

[« Previous](#)

[Next Chapter »](#)

Positioning can be tricky sometimes!

Decide which element to display in front!

Elements can overlap!

Positioning

The CSS positioning properties allow you to position an element. It can also place an element behind another, and specify what should happen when an element's content is too big.

Elements can be positioned using the top, bottom, left, and right properties. However, these properties will not work unless the position property is set first. They also work differently depending on the positioning method.

There are four different positioning methods.

Static Positioning

HTML elements are positioned static by default. A static positioned element is always positioned according to the normal flow of the page.

Static positioned elements are not affected by the top, bottom, left, and right properties.

Fixed Positioning

An element with fixed position is positioned relative to the browser window.

It will not move even if the window is scrolled:

Example

```
p.pos_fixed
{
  position:fixed;
  top:30px;
  right:5px;
}
```

[Try it yourself »](#)

Note: Internet Explorer supports the fixed value only if a !DOCTYPE is specified.

Fixed positioned elements are removed from the normal flow. The document and other elements behave like the fixed positioned element does not exist.

Fixed positioned elements can overlap other elements.

Relative Positioning

A relative positioned element is positioned relative to its normal position.

Example

```
h2.pos_left
{
position:relative;
left:-20px;
}
h2.pos_right
{
position:relative;
left:20px;
}
```

[Try it yourself »](#)

The content of a relatively positioned elements can be moved and overlap other elements, but the reserved space for the element is still preserved in the normal flow.

Example

```
h2.pos_top
{
position:relative;
top:-50px;
}
```

[Try it yourself »](#)

Relatively positioned element are often used as container blocks for absolutely positioned elements.

Absolute Positioning

An absolute position element is positioned relative to the first parent element that has a position other than static. If no such element is found, the containing block is <html>:

Example

```
h2
{
position:absolute;
left:100px;
top:150px;
}
```

[Try it yourself »](#)

Absolutely positioned elements are removed from the normal flow. The document and other elements behave like the absolutely positioned element does not exist.

Absolutely positioned elements can overlap other elements.

Overlapping Elements

When elements are positioned outside the normal flow, they can overlap other elements.

The z-index property specifies the stack order of an element (which element should be placed in front of, or behind, the others).

An element can have a positive or negative stack order:

Example

```
img
{
  position: absolute;
  left: 0px;
  top: 0px;
  z-index: -1
}
```

[Try it yourself »](#)

An element with greater stack order is always in front of an element with a lower stack order.



More Examples

[Set the shape of an element](#)

This example demonstrates how to set the shape of an element. The element is clipped into this shape, and displayed.

[How to show overflow in an element using scroll](#)

This example demonstrates how to set the overflow property to create a scroll bar when an element's content is too big to fit in a specified area.

[How to set the browser to automatically handle overflow](#)

This example demonstrates how to set the browser to automatically handle overflow.

[Change the cursor](#)

This example demonstrates how to change the cursor.

All CSS Positioning Properties

The number in the "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Property	Description	Values	CSS
<u>bottom</u>	Sets the bottom margin edge for a positioned box	auto <i>length</i> % inherit	2
<u>clip</u>	Clips an absolutely positioned element	<i>shape</i> auto inherit	2
<u>cursor</u>	Specifies the type of cursor to be displayed	<i>url</i> auto crosshair default pointer move e-resize ne-resize nw-resize n-resize se-resize sw-resize s-resize w-resize text wait help	2
<u>left</u>	Sets the left margin edge for a positioned box	auto <i>length</i> % inherit	2
<u>overflow</u>	Specifies what happens if content overflows an element's box	auto hidden scroll visible inherit	2
<u>position</u>	Specifies the type of positioning for an element	absolute fixed relative static inherit	2
<u>right</u>	Sets the right margin edge for a positioned box	auto <i>length</i> % inherit	2
<u>top</u>	Sets the top margin edge for a positioned box	auto <i>length</i> % inherit	2
<u>z-index</u>	Sets the stack order of an element	<i>number</i> auto inherit	2

CSS Float

[« Previous](#)

[Next Chapter »](#)

What is CSS Float?



With CSS float, an element can be pushed to the left or right, allowing other elements to wrap around it.

Float is very often used for images, but it is also useful when working with layouts.

How Elements Float

Elements are floated horizontally, this means that an element can only be floated left or right, not up or down.

A floated element will move as far to the left or right as it can. Usually this means all the way to the left or right of the containing element.

The elements after the floating element will flow around it.

The elements before the floating element will not be affected.

If an image is floated to the right, a following text flows around it, to the left:

Example

```
img
{
float:right;
}
```

[Try it yourself »](#)

Floating Elements Next to Each Other

If you place several floating elements after each other, they will float next to each other if there is room.

Here we have made an image gallery using the float property:

Example

```
.thumbnail
{
float:left;
width:110px;
height:90px;
margin:5px;
}
```

[Try it yourself »](#)

Turning off Float - Using Clear

Elements after the floating element will flow around it. To avoid this, use the clear property.

The clear property specifies which sides of an element other floating elements are not allowed.

Add a text line into the image gallery, using the clear property:

Example

```
.text_line
{
clear:both;
}
```

[Try it yourself »](#)



More Examples

[An image with border and margins that floats to the right in a paragraph](#)

Let an image float to the right in a paragraph. Add border and margins to the image.

[An image with a caption that floats to the right](#)

Let an image with a caption float to the right.

[Let the first letter of a paragraph float to the left](#)

Let the first letter of a paragraph float to the left and style the letter.

[Creating a horizontal menu](#)

Use float with a list of hyperlinks to create a horizontal menu.

[Creating a homepage without tables](#)

Use float to create a homepage with a header, footer, left content and main content.

All CSS Float Properties

The number in the "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Property	Description	Values	CSS
clear	Specifies which sides of an element where other floating elements are not allowed	left right both none inherit	1
float	Specifies whether or not a box should float	left right none inherit	1

CSS Horizontal Align

[« Previous](#)

[Next Chapter »](#)

In CSS, several properties are used to align elements horizontally.

Aligning Block Elements

A block element is an element that takes up the full width available, and has a line break before and after it.

Examples of block elements:

- `<h1>`
- `<p>`
- `<div>`

For aligning text, see the [CSS Text](#) chapter.

In this chapter we will show you how to horizontally align block elements for layout purposes.

Center Aligning Using the margin Property

Block elements can be aligned by setting the left and right margins to "auto".

Note: Using margin:auto will not work in Internet Explorer, **unless a !DOCTYPE is declared**.

Setting the left and right margins to auto specifies that they should split the available margin equally. The result is a centered element:

Example

```
.center
{
margin-left:auto;
margin-right:auto;
width:70%;
background-color:#b0e0e6;
}
```

[Try it yourself »](#)

Tip: Aligning has no effect if the width is 100%.

Note: In IE 5 there is a margin handling bug for block elements. To make the example above work in IE5, add some extra code. [Try it yourself](#)

Left and Right Aligning Using the position Property

One method of aligning elements is to use absolute positioning:

Example

```
.right
{
position:absolute;
right:0px;
width:300px;
background-color:#b0e0e6;
}
```

[Try it yourself »](#)

Note: Absolute positioned elements are removed from the normal flow, and can overlap elements.

Crossbrowser Compatibility Issues

When aligning elements like this, it is always a good idea to predefine margin and padding for the <body> element. This is to avoid visual differences in different browsers.

There is also another problem with IE when using the position property. If a container element (in our case <div class="container">) has a specified width, and the !DOCTYPE declaration is missing, IE will add a 17px margin on the right side. This seems to be space reserved for a scrollbar. Always set the !DOCTYPE declaration when using the position property:

Example

```
body
{
margin:0;
padding:0;
}
.container
{
position:relative;
```

```
width:100%;
}
.right
{
position:absolute;
right:0px;
width:300px;
background-color:#b0e0e6;
}
```

[Try it yourself »](#)

Left and Right Aligning Using the float Property

One method of aligning elements is to use the float property:

Example

```
.right
{
float:right;
width:300px;
background-color:#b0e0e6;
}
```

[Try it yourself »](#)

Crossbrowser Compatibility Issues

When aligning elements like this, it is always a good idea to predefine margin and padding for the <body> element. This is to avoid visual differences in different browsers.

There is also another problem with IE when using the float property. If the !DOCTYPE declaration is missing, IE will add a 17px margin on the right side. This seems to be space reserved for a scrollbar. Always set the !DOCTYPE declaration when using the float property:

Example

```
body
{
margin:0;
padding:0;
}
.right
{
float:right;
width:300px;
background-color:#b0e0e6;
}
```

[Try it yourself »](#)

CSS Pseudo-classes

[« Previous](#)

[Next Chapter »](#)

CSS pseudo-classes are used to add special effects to some selectors.

Syntax

The syntax of pseudo-classes:

```
selector:pseudo-class {property:value;}
```

CSS classes can also be used with pseudo-classes:

```
selector.class:pseudo-class {property:value;}
```

Anchor Pseudo-classes

Links can be displayed in different ways in a CSS-supporting browser:

Example

```
a:link {color:#FF0000;}          /* unvisited link */  
a:visited {color:#00FF00;}      /* visited link */  
a:hover {color:#FF00FF;}        /* mouse over link */  
a:active {color:#0000FF;}       /* selected link */
```

[Try it yourself »](#)

Note: a:hover MUST come after a:link and a:visited in the CSS definition in order to be effective!!

Note: a:active MUST come after a:hover in the CSS definition in order to be effective!!

Note: Pseudo-class names are not case-sensitive.

Pseudo-classes and CSS Classes

Pseudo-classes can be combined with CSS classes:

```
a.red:visited {color:#FF0000;}  
  
<a class="red" href="css_syntax.asp">CSS Syntax</a>
```

If the link in the example above has been visited, it will be displayed in red.

CSS - The :first-child Pseudo-class

The `:first-child` pseudo-class matches a specified element that is the first child of another element.

Note: For `:first-child` to work in IE a `<!DOCTYPE>` must be declared.

Match the first `<p>` element

In the following example, the selector matches any `<p>` element that is the first child of any element:

Example

```
<html>
<head>
<style type="text/css">
p:first-child
{
color:blue;
}
</style>
</head>

<body>
<p>I am a strong man.</p>
<p>I am a strong man.</p>
</body>
</html>
```

[Try it yourself »](#)

Match the first `<i>` element in all `<p>` elements

In the following example, the selector matches the first `<i>` element in all `<p>` elements:

Example

```
<html>
<head>
<style type="text/css">
p > i:first-child
{
font-weight:bold;
}
</style>
</head>

<body>
<p>I am a <i>strong</i> man. I am a <i>strong</i> man.</p>
<p>I am a <i>strong</i> man. I am a <i>strong</i> man.</p>
</body>
</html>
```

[Try it yourself »](#)

Match all `<i>` elements in all first child `<p>` elements

In the following example, the selector matches all `<i>` elements in `<p>` elements that are the first child of another element:

Example

```
<html>
<head>
<style type="text/css">
p:first-child i
{
color:blue;
}
</style>
</head>

<body>
<p>I am a <i>strong</i> man. I am a <i>strong</i> man.</p>
<p>I am a <i>strong</i> man. I am a <i>strong</i> man.</p>
</body>
</html>
```

[Try it yourself »](#)

CSS - The :lang Pseudo-class

The :lang pseudo-class allows you to define special rules for different languages.

Note: Internet Explorer 8 (and higher) supports the :lang pseudo-class if a [<!DOCTYPE>](#) is specified.

In the example below, the :lang class defines the quotation marks for q elements with lang="no":

Example

```
<html>
<head>
<style type="text/css">
q:lang(no) {quotes: "~" "~";}
</style>
</head>

<body>
<p>Some text <q lang="no">A quote in a paragraph</q> Some text.</p>
</body>
</html>
```

[Try it yourself »](#)



More Examples

[Add different styles to hyperlinks](#)

This example demonstrates how to add other styles to hyperlinks.

[Use of :focus](#)

This example demonstrates how to use the :focus pseudo-class.

Pseudo-classes

The "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Pseudo name	Description	CSS
:active	Adds a style to an element that is activated	1
:first-child	Adds a style to an element that is the first child of another element	2
:focus	Adds a style to an element that has keyboard input focus	2
:hover	Adds a style to an element when you mouse over it	1
:lang	Adds a style to an element with a specific lang attribute	2
:link	Adds a style to an unvisited link	1
:visited	Adds a style to a visited link	1

CSS Pseudo-elements

[« Previous](#)

[Next Chapter »](#)

CSS pseudo-elements are used to add special effects to some selectors.

Syntax

The syntax of pseudo-elements:

```
selector:pseudo-element {property:value;}
```

CSS classes can also be used with pseudo-elements:

```
selector.class:pseudo-element {property:value;}
```

The :first-line Pseudo-element

The "first-line" pseudo-element is used to add a special style to the first line of a text.

In the following example the browser formats the first line of text in a p element according to the style in the "first-line" pseudo-element (where the browser breaks the line, depends on the size of the browser window):

Example

```
p:first-line
{
  color:#ff0000;
  font-variant:small-caps;
}
```

[Try it yourself »](#)

Note: The "first-line" pseudo-element can only be used with block-level elements.

Note: The following properties apply to the "first-line" pseudo-element:

- font properties
- color properties
- background properties
- word-spacing
- letter-spacing
- text-decoration
- vertical-align
- text-transform
- line-height
- clear

The :first-letter Pseudo-element

The "first-letter" pseudo-element is used to add a special style to the first letter of a text:

Example

```
p:first-letter
{
  color:#ff0000;
  font-size:xx-large;
}
```

[Try it yourself >>](#)

Note: The "first-letter" pseudo-element can only be used with block-level elements.

Note: The following properties apply to the "first-letter" pseudo- element:

- font properties
- color properties
- background properties
- margin properties
- padding properties
- border properties
- text-decoration
- vertical-align (only if "float" is "none")
- text-transform
- line-height
- float
- clear

Pseudo-elements and CSS Classes

Pseudo-elements can be combined with CSS classes:

```
p.article:first-letter {color:#ff0000;}
```

```
<p class="article">A paragraph in an article</p>
```

The example above will display the first letter of all paragraphs with class="article", in red.

Multiple Pseudo-elements

Several pseudo-elements can also be combined.

In the following example, the first letter of a paragraph will be red, in an xx-large font size. The rest of the first line will be blue, and in small-caps. The rest of the paragraph will be the default font size and color:

Example

```
p:first-letter
{
  color:#ff0000;
  font-size:xx-large;
}
p:first-line
{
  color:#0000ff;
  font-variant:small-caps;
}
```

[Try it yourself »](#)

CSS - The :before Pseudo-element

The ":before" pseudo-element can be used to insert some content before the content of an element.

The following example inserts an image before each <h1> element:

Example

```
h1:before
{
  content:url(smiley.gif);
}
```

[Try it yourself »](#)

CSS - The :after Pseudo-element

The ":after" pseudo-element can be used to insert some content after the content of an element.

The following example inserts an image after each <h1> element:

Example

```
h1:after
```

```
{
content:url(smiley.gif);
}
```

[Try it yourself »](#)

Pseudo-elements

The "CSS" column indicates in which CSS version the property is defined (CSS1 or CSS2).

Pseudo name	Description	CSS
:after	Adds content after an element	2
:before	Adds content before an element	2
:first-letter	Adds a style to the first character of a text	1
:first-line	Adds a style to the first line of a text	1

CSS Navigation Bar

[« Previous](#)

[Next Chapter »](#)

Demo: Navigation Bar

- [HOME](#)
- [NEWS](#)
- [ARTICLES](#)
- [FORUM](#)
- [CONTACT](#)
- [ABOUT](#)

Navigation Bars

Having easy-to-use navigation is important for any web site.

With CSS you can transform boring HTML menus into good-looking navigation bars.

Navigation Bar = List of Links

A navigation bar needs standard HTML as a base.

In our examples we will build the navigation bar from a standard HTML list.

A navigation bar is basically a list of links, so using the and elements makes perfect sense:

Example

```
<ul>
<li><a href="default.asp">Home</a></li>
<li><a href="news.asp">News</a></li>
<li><a href="contact.asp">Contact</a></li>
<li><a href="about.asp">About</a></li>
</ul>
```

[Try it yourself »](#)

Now let's remove the bullets and the margins and padding from the list:

Example

```
ul
{
list-style-type:none;
margin:0;
padding:0;
}
```

[Try it yourself »](#)

Example explained:

- list-style-type:none - Removes the bullets. A navigation bar does not need list markers
- Setting margins and padding to 0 to remove browser default settings

The code in the example above is the standard code used in both vertical, and horizontal navigation bars.

Vertical Navigation Bar

To build a vertical navigation bar we only need to style the <a> elements, in addition to the code above:

Example

```
a
{
display:block;
width:60px;
}
```

[Try it yourself »](#)

Example explained:

- display:block - Displaying the links as block elements makes the whole link area clickable (not just the text), and it allows us to specify the width
- width:60px - Block elements take up the full width available by default. We want to specify a 60 px width

Tip: Also take a look at our [fully styled vertical navigation bar example](#).

Note: Always specify the width for <a> elements in a vertical navigation bar. If you omit the width, IE6 can produce unexpected results.

Horizontal Navigation Bar

There are two ways to create a horizontal navigation bar. Using **inline** or **floating** list items.

Both methods work fine, but if you want the links to be the same size, you have to use the floating method.

Inline List Items

One way to build a horizontal navigation bar is to specify the elements as inline, in addition to the "standard" code above:

Example

```
li
{
display:inline;
}
```

[Try it yourself »](#)

Example explained:

- display:inline; - By default, elements are block elements. Here, we remove the line breaks before and after each list item, to display them on one line

Tip: Also take a look at our [fully styled horizontal navigation bar example](#).

Floating List Items

In the example above the links have different widths.

For all the links to have an equal width, float the elements and specify a width for the <a> elements:

Example

```
li
{
float:left;
}
a
{
display:block;
width:60px;
}
```

[Try it yourself »](#)

Example explained:

- float:left - use float to get block elements to slide next to each other

- display:block - Displaying the links as block elements makes the whole link area clickable (not just the text), and it allows us to specify the width
- width:60px - Since block elements take up the full width available, they cannot float next to each other. We specify the width of the links to 60px

Tip: Also take a look at our [fully styled horizontal navigation bar example](#).

CSS Image Gallery

[« Previous](#)

[Next Chapter »](#)

CSS can be used to create an image gallery.



Add a description of the image here



Add a description of the image here



Add a description of the image here



Add a description of the image here

Image Gallery

The following image gallery is created with CSS:

Example

```
<html>
<head>
<style type="text/css">
div.img
{
margin:2px;
border:1px solid #0000ff;
}
```

```

height:auto;
width:auto;
float:left;
text-align:center;
}
div.img img
{
display:inline;
margin:3px;
border:1px solid #ffffff;
}
div.img a:hover img
{
border:1px solid #0000ff;
}
div.desc
{
text-align:center;
font-weight:normal;
width:120px;
margin:2px;
}
</style>
</head>
<body>

<div class="img">
  <a target="_blank" href="klematis_big.htm">
    
  </a>
  <div class="desc">Add a description of the image here</div>
</div>
<div class="img">
  <a target="_blank" href="klematis2_big.htm">
    
  </a>
  <div class="desc">Add a description of the image here</div>
</div>
<div class="img">
  <a target="_blank" href="klematis3_big.htm">
    
  </a>
  <div class="desc">Add a description of the image here</div>
</div>
<div class="img">
  <a target="_blank" href="klematis4_big.htm">
    
  </a>
  <div class="desc">Add a description of the image here</div>
</div>

</body>
</html>

```

CSS Image Opacity / Transparency

[« Previous](#)

[Next Chapter »](#)

Creating transparent images with CSS is easy.



Try it Yourself - Examples

[Creating transparent images - mouseover effect](#)

[Creating a transparent box with text on a background image](#)

Note: This is not yet a CSS standard. However, it works in all modern browsers, and is a part of the W3C CSS 3 recommendation.

Example 1 - Creating a Transparent Image

First we will show you how to create a transparent image with CSS.

Regular image:



The same image with transparency:



Look at the following source code:

```

```

Firefox uses the property **opacity:x** for transparency, while IE uses **filter:alpha(opacity=x)**.

Tip: The CSS3 syntax for transparency is **opacity:x**.

In Firefox (opacity:x) x can be a value from 0.0 - 1.0. A lower value makes the element more transparent.

In IE (filter:alpha(opacity=x)) x can be a value from 0 - 100. A lower value makes the element more transparent.

Example 2 - Image Transparency - Mouseover Effect

Mouse over the images:



The source code looks like this:

```



```

We see that the first line of the source code is similar to the source code in Example 1. In addition, we have added an onmouseover attribute and an onmouseout attribute. The onmouseover attribute defines what will happen when the mouse pointer moves over the image. In this case we want the image to NOT be transparent when we move the mouse pointer over it.

The syntax for this in Firefox is: **this.style.opacity=1** and the syntax in IE is: **this.filters.alpha.opacity=100**.

When the mouse pointer moves away from the image, we want the image to be transparent again. This is done in the onmouseout attribute.

Example 3 - Text in Transparent Box

This is some text that is placed in the transparent box. This is some text that is placed in the transparent box. This is some text that is placed in the transparent box. This is some text that is placed in the transparent box.

The source code looks like this:

```
<html>
<head>
<style type="text/css">
div.background
{
width:500px;
height:250px;
background:url(klematis.jpg) repeat;
border:2px solid black;
}
```

```

div.transbox
{
width:400px;
height:180px;
margin:30px 50px;
background-color:#ffffff;
border:1px solid black;
/* for IE */
filter:alpha(opacity=60);
/* CSS3 standard */
opacity:0.6;
}
div.transbox p
{
margin:30px 40px;
font-weight:bold;
color:#000000;
}
</style>
</head>

<body>

<div class="background">
<div class="transbox">
<p>This is some text that is placed in the transparent box.
This is some text that is placed in the transparent box.
This is some text that is placed in the transparent box.
This is some text that is placed in the transparent box.
This is some text that is placed in the transparent box.
</p>
</div>
</div>

</body>
</html>

```

First, we create a div element (class="background") with a fixed height and width, a background image, and a border. Then we create a smaller div (class="transbox") inside the first div element. This div also have a fixed width, a background image, and a border. In addition we make this div transparent.

Inside the transparent div, we add some text inside a p element.

CSS Image Sprites

[« Previous](#)

[Next Chapter »](#)

Image Sprites

An image sprite is a collection of images put into a single image.

A web page with many images can take a long time to load and generates multiple server requests.

Using image sprites will reduce the number of server requests and save bandwidth.

Image Sprites - Simple Example

Instead of using three separate images, we use this single image ("img_navsprites.gif"):



With CSS, we can show just the part of the image we need.

In the following example the CSS specifies which part of the "img_navsprites.gif" image to show:

Example

```
img.home
{
width:46px;
height:44px;
background:url(img_navsprites.gif) 0 0;
}
```

[Try it yourself >>](#)

Example explained:

- `` - Only defines a small transparent image because the src attribute cannot be empty. The displayed image will be the background image we specify in CSS
- `width:46px;height:44px;` - Defines the portion of the image we want to use
- `background:url(img_navsprites.gif) 0 0;` - Defines the background image and its position (left 0px, top 0px)

This is the easiest way to use image sprites, now we want to expand it by using links and hover effects.

Image Sprites - Create a Navigation List

We want to use the sprite image ("img_navsprites.gif") to create a navigation list.

We will use an HTML list, because it can be a link and also supports a background image:

Example

```
#navlist{position:relative;}
#navlist li{margin:0;padding:0;list-style:none;position:absolute;top:0;}
#navlist li, #navlist a{height:44px;display:block;}

#home{left:0px;width:46px;}
#home{background:url('img_navsprites.gif') 0 0;}

#prev{left:63px;width:43px;}
#prev{background:url('img_navsprites.gif') -47px 0;}

#next{left:129px;width:43px;}
```

```
#next{background:url('img_navsprites.gif') -91px 0;}
```

[Try it yourself »](#)

Example explained:

- `#navlist{position:relative;}` - position is set to relative to allow absolute positioning inside it
- `#navlist li{margin:0;padding:0;list-style:none;position:absolute;top:0;}` - margin and padding is set to 0, list-style is removed, and all list items are absolute positioned
- `#navlist li, #navlist a{height:44px;display:block;}` - the height of all the images are 44px

Now start to position and style for each specific part:

- `#home{left:0px;width:46px;}` - Positioned all the way to the left, and the width of the image is 46px
- `#home{background:url(img_navsprites.gif) 0 0;}` - Defines the background image and its position (left 0px, top 0px)
- `#prev{left:63px;width:43px;}` - Positioned 63px to the right (`#home` width 46px + some extra space between items), and the width is 43px.
- `#prev{background:url('img_navsprites.gif') -47px 0;}` - Defines the background image 47px to the right (`#home` width 46px + 1px line divider)
- `#next{left:129px;width:43px;}` - Positioned 129px to the right (start of `#prev` is 63px + `#prev` width 43px + extra space), and the width is 43px.
- `#next{background:url('img_navsprites.gif') no-repeat -91px 0;}` - Defines the background image 91px to the right (`#home` width 46px + 1px line divider + `#prev` width 43px + 1px line divider)

Image Sprites - Hover Effect

Now we want to add a hover effect to our navigation list.

Our new image ("img_navsprites_hover.gif") contains three navigation images and three images to use for hover effects:



Because this is one single image, and not six separate files, there will be **no loading delay** when a user hovers over the image.

We only add three lines of code to add the hover effect:

Example

```
#home a:hover{background: url('img_navsprites_hover.gif') 0 -45px;}
#prev a:hover{background: url('img_navsprites_hover.gif') -47px -45px;}
#next a:hover{background: url('img_navsprites_hover.gif') -91px -45px;}
```

[Try it yourself »](#)

Example explained:

- Since the list item contains a link, we can use the :hover pseudo-class
- #home a:hover{background: transparent url(img_navsprites_hover.gif) 0 -45px;} - For all three hover images we specify the same background position, only 45px further down

CSS Media Types

[« Previous](#)

[Next Chapter »](#)

Media Types allow you to specify how documents will be presented in different media. The document can be displayed differently on the screen, on the paper, with an aural browser, etc.

Media Types

Some CSS properties are only designed for a certain media. For example the "voice-family" property is designed for aural user agents. Some other properties can be used for different media types. For example, the "font-size" property can be used for both screen and print media, but perhaps with different values. A document usually needs a larger font-size on a screen than on paper, and sans-serif fonts are easier to read on the screen, while serif fonts are easier to read on paper.

The @media Rule

The @media rule allows different style rules for different media in the same style sheet.

The style in the example below tells the browser to display a 14 pixels Verdana font on the screen. But if the page is printed, it will be in a 10 pixels Times font. Notice that the font-weight is set to bold, both on screen and on paper:

```
<html>
<head>
<style>
@media screen
{
  p.test {font-family:verdana,sans-serif;font-size:14px;}
}
@media print
{
  p.test {font-family:times,serif;font-size:10px;}
}
@media screen,print
{
  p.test {font-weight:bold;}
}
</style>
</head>

<body>
....
</body>
</html>
```

See it yourself ! If you are using Mozilla/Firefox or IE 5+ and print this page, you will see that the paragraph under "Media Types" will be displayed in another font, and have a smaller font size than the rest of the text.

Different Media Types

Note: The media type names are not case-sensitive.

Media Type	Description
all	Used for all media type devices
aural	Used for speech and sound synthesizers
braille	Used for braille tactile feedback devices
embossed	Used for paged braille printers
handheld	Used for small or handheld devices
print	Used for printers
projection	Used for projected presentations, like slides
screen	Used for computer screens
tty	Used for media using a fixed-pitch character grid, like teletypes and terminals
tv	Used for television-type devices

CSS Attribute Selectors

[« Previous](#)

[Next Chapter »](#)

Style HTML Elements With Specific Attributes

It is possible to style HTML elements that have specific attributes, not just class and id.

Note: Internet Explorer 7 (and higher) supports attribute selectors only if a !DOCTYPE is specified. Attribute selection is **NOT** supported in IE6 and lower.

Attribute Selector

The example below styles all elements with a title attribute:

Example

```
[title]
{
  color:blue;
}
```

[Try it yourself »](#)

Attribute and Value Selector

The example below styles all elements with title="W3Schools":

Example

```
[title=W3Schools]
{
border:5px solid green;
}
```

[Try it yourself »](#)

Attribute and Value Selector - Multiple Values

The example below styles all elements with a title attribute that contains a specified value. This works even if the attribute has space separated values:

Example

```
[title~=hello] { color:blue; }
```

[Try it yourself »](#)

The example below styles all elements with a lang attribute that contains a specified value. This works even if the attribute has hyphen (-) separated values:

Example

```
[lang|=en] { color:blue; }
```

[Try it yourself »](#)

Styling Forms

The attribute selectors are particularly useful for styling forms without class or ID:

Example

```
input[type="text"]
{
width:150px;
display:block;
margin-bottom:10px;
background-color:yellow;
}
input[type="button"]
{
width:120px;
margin-left:35px;
display:block;
}
```

CSS Don't

[<< Previous](#)

[Next Chapter >>](#)

Here are some technologies you should try to avoid when using CSS.

Internet Explorer Behaviors

What is it? Internet Explorer 5 introduced behaviors. Behaviors are a way to add behaviors to HTML elements with the use of CSS styles.

Why avoid it? The behavior attribute is only supported by Internet Explorer.

What to use instead? Use JavaScript and the [HTML DOM](#) instead.

Example 1 - Mouseover Highlight

The following HTML file has a <style> element that defines a behavior for the <h1> element:

```
<html>
<head>
<style type="text/css">
h1 { behavior:url(behave.htc); }
</style>
</head>

<body>
<h1>Mouse over me!!!</h1>
</body>
</html>
```

The XML document "behave.htc" is shown below:

Example (IE 5+ Only)

The behavior file contains a JavaScript and event handlers for the elements.

```
<attach for="element" event="onmouseover" handler="hig_lite" />
<attach for="element" event="onmouseout" handler="low_lite" />

<script type="text/javascript">
function hig_lite()
{
element.style.color='red';
}

function low_lite()
{
element.style.color='blue';
}
</script>
```

[Try it yourself »](#)

Example 2 - Typewriter Simulation

The following HTML file has a <style> element that defines a behavior for elements with an id of "typing":

```
<html>
<head>
<style type="text/css">
#typing
{
behavior:url(behave_typing.htc);
font-family:"courier new";
}
</style>
</head>

<body>
<span id="typing" speed="100">IE5 introduced DHTML behaviors.
Behaviors are a way to add DHTML functionality to HTML elements
with the ease of CSS.<br /><br />How do behaviors work?<br />
By using XML we can link behaviors to any element in a web page
and manipulate that element.</p>
</span>
</body>
</html>
```

The XML document "typing.htc" is shown below:

Example (IE 5+ Only)

```
<attach for="window" event="onload" handler="beginTyping" />
<method name="type" />

<script type="text/javascript">
var i,text1,text2,textLength,t;

function beginTyping()
{
i=0;
text1=element.innerText;
textLength=text1.length;
element.innerText="";
text2="";
t=window.setInterval(element.id+".type()",speed);
}

function type()
{
text2=text2+text1.substring(i,i+1);
element.innerText=text2;
i=i+1;
if (i==textLength)
{
clearInterval(t);
}
```

```
}  
}  
</script>
```

CSS Examples

[« Previous](#)

[Next Chapter »](#)

CSS Backgrounds

[Set the background color of a page](#)
[Set the background color of different elements](#)
[Set an image as the background of a page](#)
[Bad background image](#)
[How to repeat a background image only horizontally](#)
[How to position a background image](#)
[A fixed background image \(this image will not scroll with the rest of the page\)](#)
[All the background properties in one declaration](#)
[Advanced background example](#)

Background properties explained

CSS Text

[Set the text color of different elements](#)
[Align the text](#)
[Remove the line under links](#)
[Decorate the text](#)
[Control the letters in a text](#)
[Indent text](#)
[Specify the space between characters](#)
[Specify the space between lines](#)
[Set the text direction of an element](#)
[Increase the white space between words](#)
[Disable text wrapping inside an element](#)
[Vertical alignment of an image inside text](#)

Text properties explained

CSS Fonts

[Set the font of a text](#)
[Set the size of the font](#)
[Set the size of the font in px](#)
[Set the size of the font in em](#)
[Set the size of the font in percent and em](#)
[Set the style of the font](#)
[Set the variant of the font](#)
[Set the boldness of the font](#)
[All the font properties in one declaration](#)

Font properties explained

CSS Links

[Add different colors to visited/unvisited links](#)
[Use of text-decoration on links](#)
[Specify a background color for links](#)
[Add other styles to hyperlinks](#)
[Advanced - Create link boxes](#)

Link properties explained

CSS Lists

[All the different list item markers in lists](#)
[Set an image as the list-item marker](#)
[Set an image as the list-item marker - Crossbrowser solution](#)
[All list properties in one declaration](#)

List properties explained

CSS Tables

[Specify a black border for table, th, and td elements](#)
[Use of border-collapse](#)
[Specify the width and height of a table](#)
[Set the horizontal alignment of content \(text-align\)](#)
[Set the vertical alignment of content \(vertical-align\)](#)
[Specify the padding for th and td elements](#)
[Specify the color of the table borders](#)
[Set the position of the table caption](#)
[Create a fancy table](#)

Table properties explained

CSS Box Model

[Specify an element with a total width of 250px](#)
[Specify an element with a total width of 250px - Crossbrowser solution](#)

Box model explained

CSS Border

[Set the width of the four borders](#)
[Set the width of the top border](#)
[Set the width of the bottom border](#)

[Set the width of the left border](#)
[Set the width of the right border](#)

[Set the style of the four borders](#)
[Set the style of the top border](#)
[Set the style of the bottom border](#)
[Set the style of the left border](#)
[Set the style of the right border](#)

[Set the color of the four borders](#)
[Set the color of the top border](#)
[Set the color of the bottom border](#)
[Set the color of the left border](#)
[Set the color of the right border](#)

[All the border properties in one declaration](#)

[Set different borders on each side](#)
[All the top border properties in one declaration](#)
[All the bottom border properties in one declaration](#)
[All the left border properties in one declaration](#)
[All the right border properties in one declaration](#)

Border properties explained

CSS Outline

[Draw a line around an element \(outline\)](#)
[Set the style of an outline](#)
[Set the color of an outline](#)
[Set the width of an outline](#)

Outline properties explained

CSS Margin

[Specify margins for an element](#)
[The margin shorthand property](#)
[Set the top margin of a text using a cm value](#)
[Set the bottom margin of a text using a percent value](#)
[Set the left margin of a text using a cm value](#)

Margin properties explained

CSS Padding

[Set the left padding of an element](#)
[Set the right padding of an element](#)
[Set the top padding of an element](#)
[Set the bottom padding of an element](#)
[All the padding properties in one declaration](#)

Padding properties explained

CSS Grouping and Nesting

[Group selectors](#)

[Nested \(descendant\) selectors](#)

Grouping and nesting explained

CSS Dimension

[Set the height of an image using a pixel value](#)

[Set the height of an image using percent](#)

[Set the width of an element using a pixel value](#)

[Set the width of an element using percent](#)

[Set the maximum height of an element](#)

[Set the maximum width of an element using a pixel value](#)

[Set the maximum width of an element using percent](#)

[Set the minimum height of an element](#)

[Set the minimum width of an element using a pixel value](#)

[Set the minimum width of an element using percent](#)

Dimension properties explained

CSS Display

[How to hide an element \(visibility:hidden\)](#)

[How to not display an element \(display:none\)](#)

[How to display an element as an inline element](#)

[How to display an element as a block element](#)

[How to make a table element collapse](#)

Display properties explained

CSS Positioning

[Position an element relative to the browser window](#)

[Position an element relative to its normal position](#)

[Position an element with an absolute value](#)

[Overlapping elements](#)

[Set the shape of an element](#)

[How to create a scroll bar when an element's content is too big to fit](#)

[How to set the browser to automatically handle overflow](#)

[Set the top edge of an image using a pixel value](#)

[Set the bottom edge of an image using a pixel value](#)

[Set the left edge of an image using a pixel value](#)

[Set the right edge of an image using a pixel value](#)

[Change the cursor](#)

Positioning properties explained

CSS Floating

[A simple use of the float property](#)

[An image with border and margins that floats to the right in a paragraph](#)

[An image with a caption that floats to the right](#)

[Let the first letter of a paragraph float to the left](#)

[Create an image gallery with the float property](#)

[Turn of float - with the clear property](#)

[Creating a horizontal menu](#)

[Creating a homepage without tables](#)

Float properties explained

CSS Aligning Elements

[Center aligning with margin](#)

[Left/Right aligning with position](#)

[Left/Right aligning with position - Crossbrowser solution](#)

[Left/Right aligning with float](#)

[Left/Right aligning with float - Crossbrowser solution](#)

Align properties explained

CSS Generated Content

[Insert the URL in parenthesis after each link with the content property](#)

[Numbering sections and sub-sections with "Section 1", "1.1", "1.2", etc.](#)

[Specify the quotation marks with the quotes property](#)

CSS Pseudo-classes

[Add different colors to a hyperlink](#)

[Add other styles to hyperlinks](#)

[Use of :focus](#)

[:first-child - match the first p element](#)

[:first-child - match the first i element in all p elements](#)

[:first-child - Match all i elements in all first child p elements](#)

[Use of :lang](#)

Pseudo-classes explained

CSS Pseudo-elements

[Make the first letter special in a text](#)

[Make the first line special in a text](#)

[Make the first letter and first line special](#)

[Use :before to insert some content before an element](#)

[Use :after to insert some content after an element](#)

Pseudo-elements explained

CSS Navigation Bars

[Fully styled vertical navigation bar](#)

[Fully styled horizontal navigation bar](#)

[Navigation bars explained](#)

CSS Image Gallery

[Image gallery](#)

[Image gallery explained](#)

CSS Image Opacity

[Creating transparent images - mouseover effect](#)

[Creating a transparent box with text on a background image](#)

[Image opacity explained](#)

CSS Image Sprites

[An image sprite](#)

[An image sprite - a navigation list](#)

[An image sprite with hover effect](#)

[Image sprites explained](#)

CSS Attribute Selectors

[Select elements with a title attribute](#)

[Select elements with title=a specific value](#)

[Select elements with title=a specific value \(even if the attribute has space-separated values\)](#)

[Select elements with title=a specific value \(even if the attribute has hyphen-separated values\)](#)

[Attribute selectors explained](#)